

## ✓ Experiment No. 1: Data Pipeline & Preprocessing

Student Name

Vinay Yadav [23101B0020]

---

Practical Name

Data Pipeline and Preprocessing for Crop Yield Prediction Dataset

---

## Resources Required

### Software

- Google Colab / Jupyter Notebook
- Python 3.10+
- Pandas
- NumPy
- Scikit-learn
- Joblib

### Dataset

- Crop Yield Prediction Dataset (`crop_yield(1).csv`)

### Work Link

- My Github Profile <https://github.com/Vinay-Yadav-07/Data-Pipeline-and-Preprocessing-for-Crop-Yield-Prediction-Dataset>
- 

## Objectives

- Load and explore the Crop Yield dataset.
- Identify and handle missing values and duplicate records.
- Perform feature engineering by creating meaningful agricultural features.
- Separate numerical and categorical features.
- Build a reusable preprocessing pipeline using Scikit-learn.
- Split the dataset into Training, Validation, and Testing sets.
- Apply preprocessing techniques such as scaling and one-hot encoding.
- Save the preprocessing pipeline and processed datasets for future machine learning tasks.

---

# Theory

Data preprocessing is one of the most important phases of the Machine Learning lifecycle. Real-world datasets often contain missing values, duplicate records, inconsistent formats, and categorical data that cannot be directly used by machine learning algorithms. Proper preprocessing improves data quality and helps build accurate, reliable, and efficient machine learning models.

A **data preprocessing pipeline** automates all preprocessing tasks and ensures consistency, reproducibility, scalability, and easier deployment. Instead of manually applying preprocessing steps every time, a pipeline performs the same sequence of operations on both training and future datasets.

In this experiment, the **Crop Yield Prediction Dataset** is used to demonstrate the preprocessing workflow. The dataset contains agricultural information such as region, soil type, crop type, rainfall, temperature, fertilizer usage, irrigation, weather conditions, and harvesting duration. These features influence the crop yield and require preprocessing before model training.

The preprocessing process includes:

- Loading and exploring the dataset.
- Checking and handling missing values.
- Detecting and removing duplicate records.
- Performing feature engineering by creating new features such as Crop Duration, Rainfall Level, and Temperature Level.
- Separating numerical and categorical features.
- Scaling numerical features using **StandardScaler**.
- Encoding categorical features using **One-Hot Encoding**.
- Splitting the dataset into **Training (70%)**, **Validation (15%)**, and **Testing (15%)** datasets.
- Saving the preprocessing pipeline and processed datasets for future use.

A reusable preprocessing pipeline ensures that the same transformations are consistently applied during both model training and deployment, making the machine learning workflow more efficient and maintainable.

---

## Expected Outcome

After completing this experiment, the preprocessing pipeline will:

- Successfully clean the dataset.
- Generate new engineered features.
- Convert categorical variables into numerical format.

- Scale numerical features.
- Split the dataset into training, validation, and testing sets.
- Save the preprocessing pipeline (`crop_preprocessing_pipeline.pkl`).
- Save the processed datasets for future machine learning experiments.

## Step 1: Upload the Dataset

```
from google.colab import files
```

```
uploaded = files.upload()
```

[Choose files](#) No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.  
Saving crop\_yield.csv to crop\_yield.csv

## Step 2: Load the Dataset

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the dataset
```

```
df = pd.read_csv("crop_yield.csv")
```

```
# Display the first 5 rows
```

```
df.head()
```

|   | Region | Soil_Type | Crop    | Rainfall_mm | Temperature_Celsius | Fertilizer_U |
|---|--------|-----------|---------|-------------|---------------------|--------------|
| 0 | West   | Sandy     | Cotton  | 897.077239  | 27.676966           | F            |
| 1 | South  | Clay      | Rice    | 992.673282  | 18.026142           | T            |
| 2 | North  | Loam      | Barley  | 147.998025  | 29.794042           | F            |
| 3 | North  | Sandy     | Soybean | 986.866331  | 16.644190           | F            |
| 4 | South  | Silt      | Wheat   | 730.379174  | 31.620687           | T            |

## Step 3: Explore the Dataset

```
# Display the first 5 rows
```

```
print("First 5 Rows:")
```

```
display(df.head())
```

```
# Display the shape of the dataset
```

```
print("\nShape of Dataset:")
```

```
print(df.shape)
```

```
# Display column names
```

```
print("\nColumn Names:")
```

```
print(df.columns)
```

```
# Display dataset information
print("\nDataset Information:")
df.info()
```

```
# Display summary statistics
print("\nSummary Statistics:")
display(df.describe())
```

First 5 Rows:

|   | Region | Soil_Type | Crop    | Rainfall_mm | Temperature_Celsius | Fertilizer_U |
|---|--------|-----------|---------|-------------|---------------------|--------------|
| 0 | West   | Sandy     | Cotton  | 897.077239  | 27.676966           | F            |
| 1 | South  | Clay      | Rice    | 992.673282  | 18.026142           | T            |
| 2 | North  | Loam      | Barley  | 147.998025  | 29.794042           | F            |
| 3 | North  | Sandy     | Soybean | 986.866331  | 16.644190           | F            |
| 4 | South  | Silt      | Wheat   | 730.379174  | 31.620687           | T            |

Shape of Dataset:

(1000000, 10)

Column Names:

```
Index(['Region', 'Soil_Type', 'Crop', 'Rainfall_mm', 'Temperature_Celsius',  
      'Fertilizer_Used', 'Irrigation_Used', 'Weather_Condition',  
      'Days_to_Harvest', 'Yield_tons_per_hectare'],  
      dtype='object')
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 1000000 entries, 0 to 999999

Data columns (total 10 columns):

| #   | Column                 | Non-Null Count   | Dtype   |
|-----|------------------------|------------------|---------|
| --- | -----                  | -----            | -----   |
| 0   | Region                 | 1000000 non-null | object  |
| 1   | Soil_Type              | 1000000 non-null | object  |
| 2   | Crop                   | 1000000 non-null | object  |
| 3   | Rainfall_mm            | 1000000 non-null | float64 |
| 4   | Temperature_Celsius    | 1000000 non-null | float64 |
| 5   | Fertilizer_Used        | 1000000 non-null | bool    |
| 6   | Irrigation_Used        | 1000000 non-null | bool    |
| 7   | Weather_Condition      | 1000000 non-null | object  |
| 8   | Days_to_Harvest        | 1000000 non-null | int64   |
| 9   | Yield_tons_per_hectare | 1000000 non-null | float64 |

dtypes: bool(2), float64(3), int64(1), object(4)

memory usage: 62.9+ MB

Summary Statistics:

|              | Rainfall_mm    | Temperature_Celsius | Days_to_Harvest | Yield_tons_per_h |
|--------------|----------------|---------------------|-----------------|------------------|
| <b>count</b> | 1000000.000000 | 1000000.000000      | 1000000.000000  | 1000000.         |
| <b>mean</b>  | 549.981901     | 27.504965           | 104.495025      | 4.               |
| <b>std</b>   | 259.851320     | 7.220608            | 25.953412       | 1.               |
| <b>min</b>   | 100.000896     | 15.000034           | 60.000000       | -1.              |
| <b>25%</b>   | 324.891090     | 21.254502           | 82.000000       | 3.               |
| <b>50%</b>   | 550.124061     | 27.507365           | 104.000000      | 4.               |
| <b>75%</b>   | 774.738520     | 33.753267           | 127.000000      | 5.               |
| <b>max</b>   | 999.998098     | 39.999997           | 149.000000      | 9.               |

## Step 4: Check Missing Values

```
# Check missing values in each column
missing_values = df.isnull().sum()

print("Missing Values in Each Column:")
print(missing_values)

# Total missing values in the dataset
print("\nTotal Missing Values:", missing_values.sum())
```

Missing Values in Each Column:

|                        |   |
|------------------------|---|
| Region                 | 0 |
| Soil_Type              | 0 |
| Crop                   | 0 |
| Rainfall_mm            | 0 |
| Temperature_Celsius    | 0 |
| Fertilizer_Used        | 0 |
| Irrigation_Used        | 0 |
| Weather_Condition      | 0 |
| Days_to_Harvest        | 0 |
| Yield_tons_per_hectare | 0 |

dtype: int64

Total Missing Values: 0

## Step 5: Check Duplicate Records

```
# Check duplicate records
duplicate_rows = df.duplicated().sum()

print("Total Duplicate Records:", duplicate_rows)
```

Total Duplicate Records: 0

## Step 6: Handle Missing Values

```
# Check if there are any missing values
if df.isnull().sum().sum() == 0:
    print("No missing values found. No handling is required.")
else:
    # Fill numerical missing values with median
    numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
    df[numerical_cols] = df[numerical_cols].fillna(df[numerical_cols].median)

    # Fill categorical missing values with mode
    categorical_cols = df.select_dtypes(include=['object', 'bool']).columns
    for col in categorical_cols:
        df[col] = df[col].fillna(df[col].mode()[0])

    print("Missing values handled successfully.")
```

No missing values found. No handling is required.

## Step 7: Handle Duplicate Records

```
# Remove duplicate records (if any)
before_rows = df.shape[0]

df = df.drop_duplicates()

after_rows = df.shape[0]

print("Rows before removing duplicates :", before_rows)
print("Rows after removing duplicates :", after_rows)
print("Duplicates removed :", before_rows - after_rows)
```

```
Rows before removing duplicates : 1000000
Rows after removing duplicates  : 1000000
Duplicates removed : 0
```

## Step 8: Feature Engineering

```
# =====
# Step 8.1: Create Crop Duration
# =====

df["Crop_Duration"] = df["Days_to_Harvest"].apply(
    lambda x: "Short" if x < 90 else
              "Medium" if x <= 120 else
              "Long"
)

# Display first 5 rows
df[["Days_to_Harvest", "Crop_Duration"]].head()
```

|   | Days_to_Harvest | Crop_Duration |
|---|-----------------|---------------|
| 0 | 122             | Long          |
| 1 | 140             | Long          |
| 2 | 106             | Medium        |
| 3 | 146             | Long          |
| 4 | 110             | Medium        |

```
# =====
# Step 8.2: Create Rainfall Level
# =====

df["Rainfall_Level"] = df["Rainfall_mm"].apply(
    lambda x: "Low" if x < 400 else
```

```

        "Medium" if x <= 700 else
        "High"
    )

# Display first 5 rows
df[["Rainfall_mm", "Rainfall_Level"]].head()

```

|   | Rainfall_mm | Rainfall_Level |
|---|-------------|----------------|
| 0 | 897.077239  | High           |
| 1 | 992.673282  | High           |
| 2 | 147.998025  | Low            |
| 3 | 986.866331  | High           |
| 4 | 730.379174  | High           |

```

# =====
# Step 8.3: Create Temperature Level
# =====

df["Temperature_Level"] = df["Temperature_Celsius"].apply(
    lambda x: "Low" if x < 20 else
              "Moderate" if x <= 30 else
              "High"
)

# Display first 5 rows
df[["Temperature_Celsius", "Temperature_Level"]].head()

```

|   | Temperature_Celsius | Temperature_Level |
|---|---------------------|-------------------|
| 0 | 27.676966           | Moderate          |
| 1 | 18.026142           | Low               |
| 2 | 29.794042           | Moderate          |
| 3 | 16.644190           | Low               |
| 4 | 31.620687           | High              |

## Step 9: Select Features and Target Variable

```

# Select input features
features = [
    'Region',
    'Soil_Type',
    'Crop',
    'Rainfall_mm',
    'Temperature_Celsius',
    'Fertilizer_Used',

```



```

        'Irrigation_Used',
        'Weather_Condition',
        'Days_to_Harvest',
        'Crop_Duration',
        'Rainfall_Level',
        'Temperature_Level'
    ]

    # Select target variable
    target = 'Yield_tons_per_hectare'

    # Create feature matrix (X) and target vector (y)
    X = df[features]
    y = df[target]

    print("Features Shape:", X.shape)
    print("Target Shape:", y.shape)

```

Features Shape: (1000000, 12)  
Target Shape: (1000000,)

## Step 10: Separate Numerical and Categorical Columns

```

# Numerical columns
num_cols = [
    'Rainfall_mm',
    'Temperature_Celsius',
    'Days_to_Harvest'
]

# Categorical columns
cat_cols = [
    'Region',
    'Soil_Type',
    'Crop',
    'Weather_Condition',
    'Fertilizer_Used',
    'Irrigation_Used',
    'Crop_Duration',
    'Rainfall_Level',
    'Temperature_Level'
]

print("Numerical Columns:")
print(num_cols)

print("\nCategorical Columns:")
print(cat_cols)

```

Numerical Columns:  
['Rainfall\_mm', 'Temperature\_Celsius', 'Days\_to\_Harvest']

Categorical Columns:  
['Region', 'Soil\_Type', 'Crop', 'Weather\_Condition', 'Fertilizer\_Used', 'Irrigation\_Used', 'Crop\_Duration', 'Rainfall\_Level', 'Temperature\_Level']

## Step 11: Build the Preprocessing Pipeline

```
# =====  
# Step 11.1: Import Required Libraries  
# =====  
  
from sklearn.pipeline import Pipeline  
from sklearn.compose import ColumnTransformer  
from sklearn.impute import SimpleImputer  
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
# =====  
# Step 11.2: Create Numerical Pipeline  
# =====  
  
numeric_pipeline = Pipeline([  
    ('imputer', SimpleImputer(strategy='median')),  
    ('scaler', StandardScaler())  
])  
  
print("Numerical Pipeline Created Successfully!")
```

Numerical Pipeline Created Successfully!

```
# =====  
# Step 11.3: Create Categorical Pipeline  
# =====  
  
categorical_pipeline = Pipeline([  
    ('imputer', SimpleImputer(strategy='most_frequent')),  
    ('encoder', OneHotEncoder(handle_unknown='ignore'))  
])  
  
print("Categorical Pipeline Created Successfully!")
```

Categorical Pipeline Created Successfully!

```
# =====  
# Step 11.4: Create Preprocessing Pipeline  
# =====  
  
preprocessor = ColumnTransformer([  
    ('num', numeric_pipeline, num_cols),  
    ('cat', categorical_pipeline, cat_cols)  
])  
  
print("Preprocessing Pipeline Created Successfully!")
```

Preprocessing Pipeline Created Successfully!

## Step 12: Split the Dataset (Train, Validation, Test)

```
# =====  
# Step 12.1: Import train_test_split  
# =====  
  
from sklearn.model_selection import train_test_split
```

```
# =====  
# Step 12.2: Split Dataset into  
# Training, Validation and Testing Sets  
# =====  
  
# Split into Training (70%) and Temporary (30%)  
X_train, X_temp, y_train, y_temp = train_test_split(  
    X,  
    y,  
    test_size=0.30,  
    random_state=42  
)  
  
# Split Temporary into Validation (15%) and Testing (15%)  
X_val, X_test, y_val, y_test = train_test_split(  
    X_temp,  
    y_temp,  
    test_size=0.50,  
    random_state=42  
)  
  
# Display the shape of each dataset  
print("Training Set :", X_train.shape)  
print("Validation Set :", X_val.shape)  
print("Testing Set :", X_test.shape)
```

```
Training Set : (700000, 12)  
Validation Set : (150000, 12)  
Testing Set : (150000, 12)
```

## Step 13: Apply the Preprocessing Pipeline

```
# =====  
# Step 13: Apply the Preprocessing Pipeline  
# =====  
  
# Apply preprocessing pipeline  
X_train_processed = preprocessor.fit_transform(X_train)  
X_val_processed = preprocessor.transform(X_val)  
X_test_processed = preprocessor.transform(X_test)  
  
# Display the shapes of processed datasets  
print("Processed Training Set Shape :", X_train_processed.shape)
```

```
print("Processed Validation Set Shape :", X_val_processed.shape)
print("Processed Testing Set Shape :", X_test_processed.shape)
```

```
Processed Training Set Shape : (700000, 35)
Processed Validation Set Shape : (150000, 35)
Processed Testing Set Shape : (150000, 35)
```

## Step 14: Save the Preprocessing Pipeline

```
# =====
# Step 14: Save the Preprocessing Pipeline
# =====

import joblib

# Save the preprocessing pipeline
joblib.dump(preprocessor, "crop_preprocessing_pipeline.pkl")

print("Preprocessing Pipeline Saved Successfully!")
```

```
Preprocessing Pipeline Saved Successfully!
```

## Step 15: Save the Processed Datasets

```
# =====
# Step 15: Save Processed Datasets
# =====

import pandas as pd

# Convert processed data to DataFrames
X_train_df = pd.DataFrame(X_train_processed.toarray() if hasattr(X_train_pro
X_val_df = pd.DataFrame(X_val_processed.toarray() if hasattr(X_val_processed
X_test_df = pd.DataFrame(X_test_processed.toarray() if hasattr(X_test_proces

# Save processed datasets
X_train_df.to_csv("X_train_processed.csv", index=False)
X_val_df.to_csv("X_val_processed.csv", index=False)
X_test_df.to_csv("X_test_processed.csv", index=False)

print("Processed datasets saved successfully!")
```

```
Processed datasets saved successfully!
```

# Conclusion

In this experiment, a reusable and efficient data preprocessing pipeline was successfully developed for the **Crop Yield Prediction Dataset**. The dataset was loaded, explored, and checked for missing values and duplicate records. Since no

missing values or duplicate records were found, the dataset was already clean for further processing.

Feature engineering was performed by creating new features such as **Crop Duration**, **Rainfall Level**, and **Temperature Level** to improve data representation. The numerical and categorical features were identified and processed using **StandardScaler** and **OneHotEncoder** through a **Scikit-learn preprocessing pipeline**. Finally, the dataset was divided into **training, validation, and testing sets**, and the preprocessing pipeline along with the processed datasets was saved for future use.